# Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

---

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

## 0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

The concepts that are going to help me with this project are, most importantly, the previous projects we have done before, as well as implementations of if statements and loops. I believe the most important thing will be dataclasses and lists as my project will rely heavily on storing data in these.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think the easiest part for me will be setting up the necessary dataclasses for my website. Dataclasses are very easy for me to create and define, and quite honestly, they are fun too.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I believe that what will take the most effort is setting up the different routes and pages and figuring out how the page will take in an input and return a desired output. I might use buttons for this project, which will probably be really difficult to figure out too.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

My phone and bed will definitely be a distraction, I tend to take too many naps. I will try to minimize this by, firstly, not getting into my bed before actual bed time, and also putting my phone away far from my desk when I begin to work on my project. I will be using the given LLM for this project but also will utilize office hours and past assignments for help.

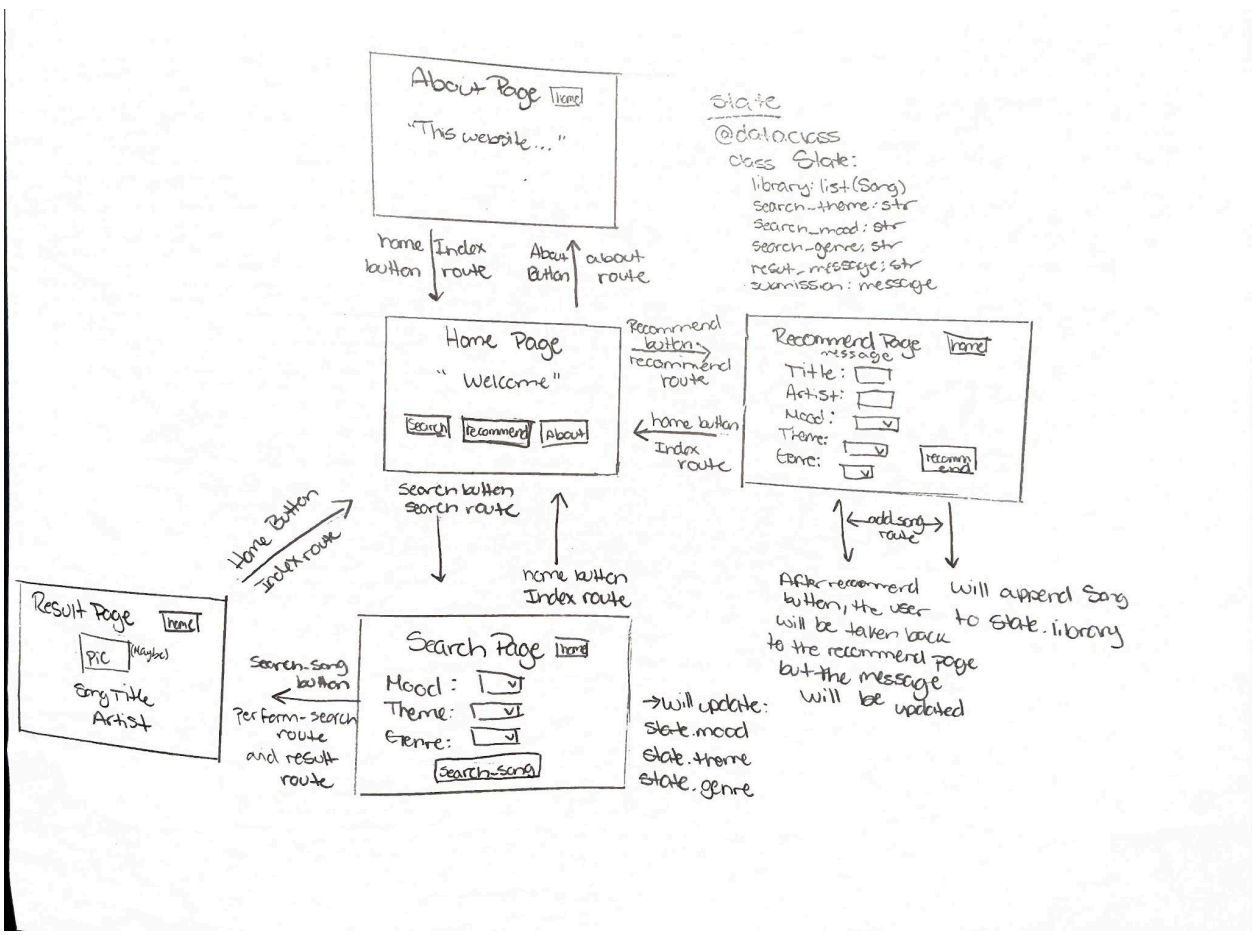
# 1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.


About Page
Home
"This website..."
State
@dataclass
class State:
library: list(Song)
search_theme: str
search_mood: str
search_genre: str
result_message: str
submission: message
home button
Index route
About Button
about route
Home Page
" Welcome"
Search
recommend
About
Recommend button
recommend route
Recommend Page
message
Home
Title:
Artist:
Mood:
Theme:
Genre:
recommend
home button
Index route
Home Button
Index route
Search button
search route
home button
Index route
addsong route
After recommend button, the user will be taken back to the recommend page but the message will be updated
Will append Song to State.library
Result Page
Home
PIC
(Maybe)
Song Title
Artist
Search Page
Home
Mood:
Theme:
Genre:
search-song
Search-song button
Perform-search route and result route
→Will update:
state.mood
state.theme
state.genre

# 2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

| **Requirement** | **Implementation** |
|---|---|
| **Purpose/Functionality**<br><br>A 1-2 sentence description of the purpose or functionality of your site | The purpose of this website is to provide users a tool to get a song recommendation based on a theme or mood they have in mind. This website will focus on alternative songs, but different genres in that umbrella will be covered. |
| **State Dataclass Attributes**<br><br>Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.<br><br>Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site. | @dataclass<br>class State:<br>library:list[Song]<br>- The library attribute is a list of songs (Datclass Song) that the program has stored which will be used for matching inputs with outputs. This will be modified through the add_song route<br>search_theme:str<br>- The search_theme attribute is the current theme selected by the user. It is modified through the perform_search route<br>search_mood:str<br>- The search_mood attribute is the current moods selected by the user. It is modified through perform_search route<br>search_genre:str<br>- The search_genre attribute is the current genre selected by the user. It is modified through the perform_search route.<br>submission_message: str<br>- attribute pertaining to the message given to the user after the submission in recommendation has been made. Updated by add_song route |

| | |
|---|---|
| **Second Dataclass**<br><br>Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified) | @dataclass<br>class Song:<br>name:str<br>artist: str<br>theme: str<br>mood:str<br>genre: str<br>Some of these will be already defined in the code but new ones may be made through the add_song route. |
| **Routes** (minimum 7)<br><br>The names of your route functions | index(state: State) - Displays the home page that will have navigation buttons.<br>search(state: State) - Displays the search page which allows the users to choose their inputs.<br>result(state: State) - Displays the result page<br>recommend(state: State) - Displays the recommend page<br>perform_search(state: State, theme: str, mood: str, genre: str) - Updates the values for state.search_theme, state.search_mood, and state.search_genre based on the boxes that the user checked<br>add_song(state: State, name: str, artist: str, theme: str, mood: str, genre: str) - this adds (appends) a new Song instance to state.library<br>about_page(state: State) - displays the about page |
| **Pages** (minimum 5)<br><br>A brief description of the pages in your site | Home: The home page will be the very first page the user will see. It will have some text welcoming the user to the site and three buttons. The buttons will be “Search”, “Recommend”, and “About”<br><br>Search Page: The search page is the page in which the user will be given different select boxes they can mark depending on what song they are looking for. After selecting specific attributes, the user will then click search song |

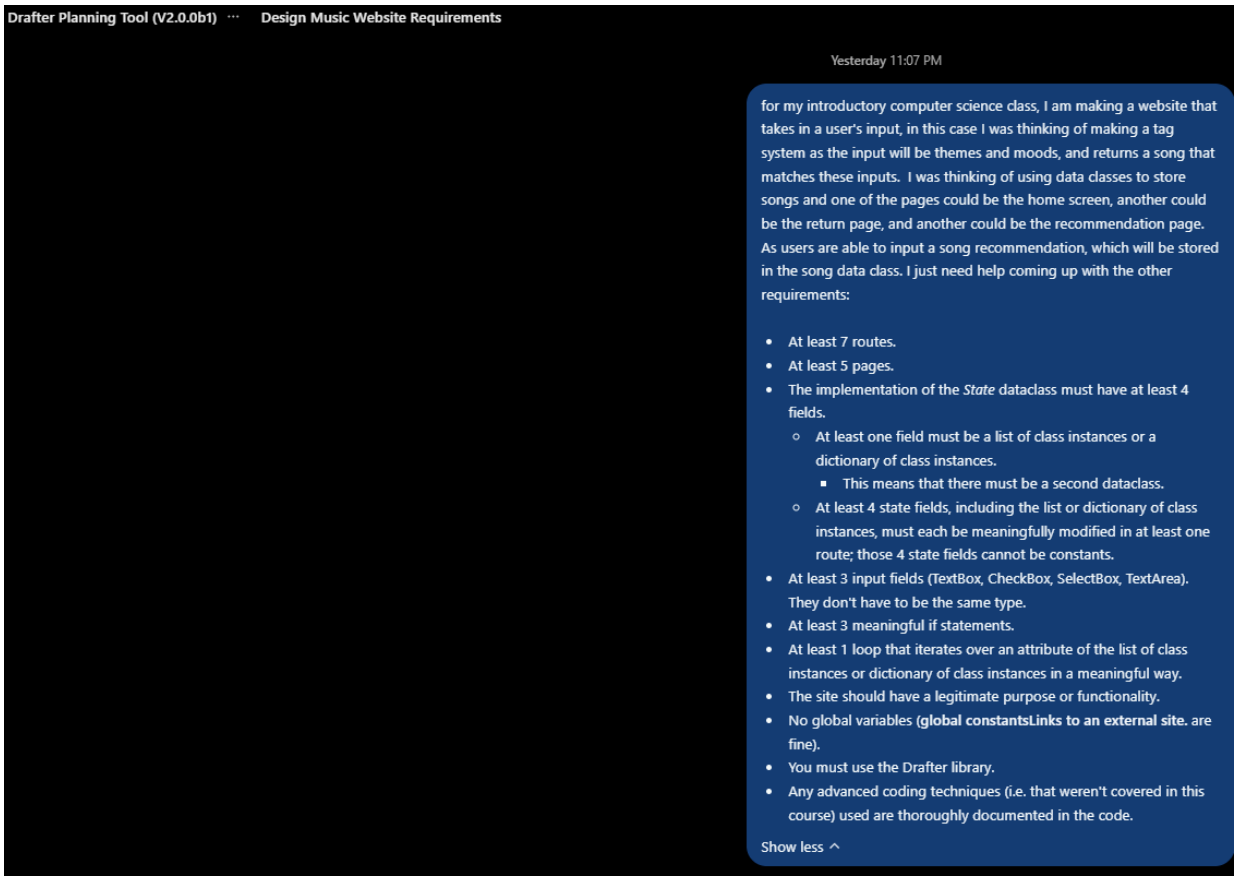
| | |
|---|---|
| | Song Page: The song page is the page that will be given to the user as an output in response to their input, it will contain a song recommended based on their matched input and the image of the album cover the song belongs to.<br><br>Recommendation Page: The recommendation page is the page that will result if the user presses the recommend button on the home page. In this page a user is able to input a song. There will be a title box in which the title of the song will be in and then checkboxes that the user will have to click to indicate the song's genre, mood, and theme.<br><br>About Page:The about page will be the page that will pop up when the user clicks the about button, it will simply contain text explaining how the website came to be and what it is about<br><br>*all these pages will have a home button to return to the home page |
| **User Input** (minimum 3)<br><br>Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes<br><br>For more information on these input types, see Drafter Documentation. | The way the user will put input is through a series of selection boxes (dropdown).<br>Each of these attributes should have one selection made<br>Theme:<br>boxes that represent a theme, the user can check one of these.<br>Options in set:<br>Romantic<br>Heartbreak<br>Rebellion<br>Isolation<br>Self-Discovery<br>Death |

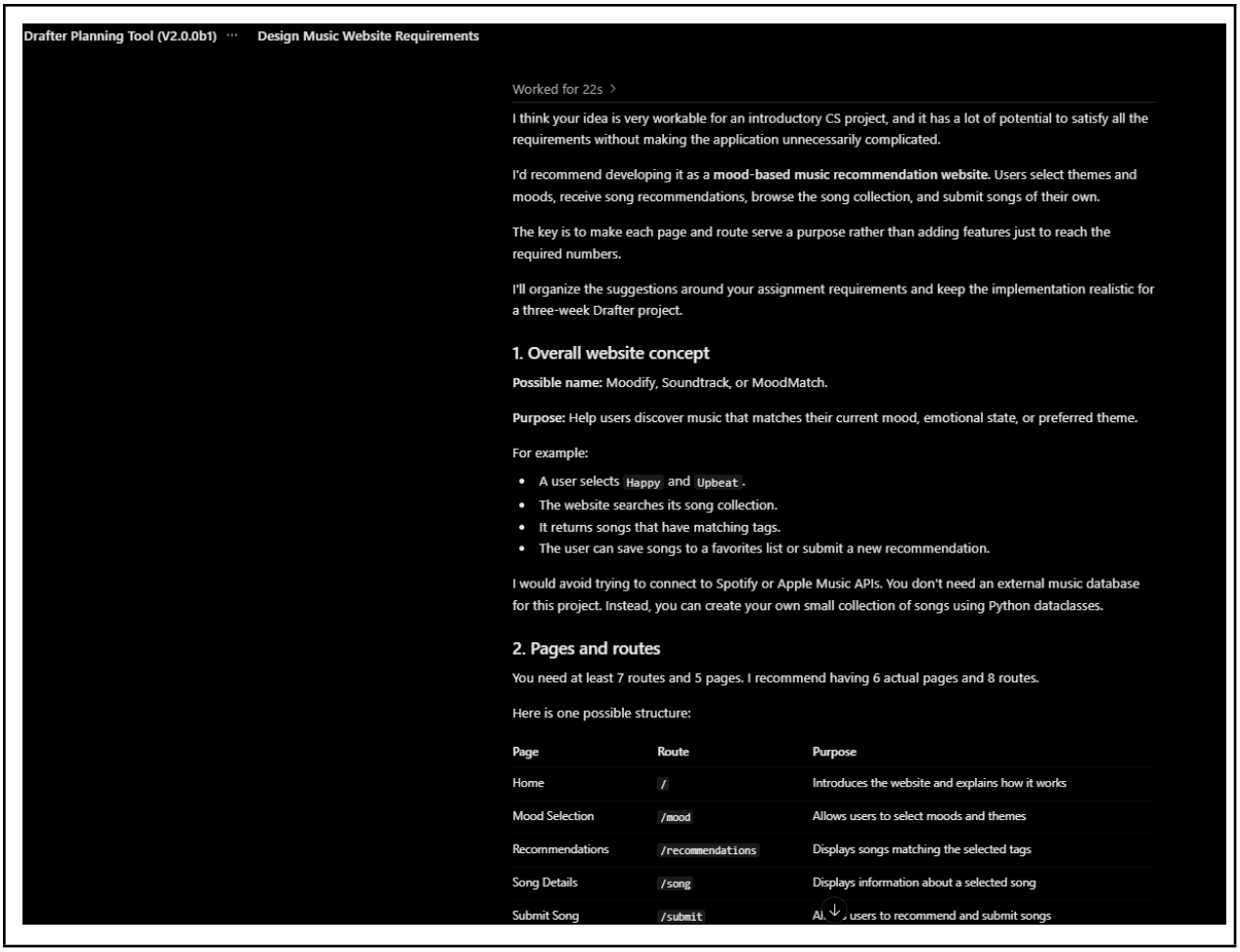
| | |
|---|---|
| | Mood:<br>boxes that represent a mood, the user can select one of these.<br>Options in set:<br>Happy<br>Sad<br>Slow<br>Upbeat<br>Dark<br>Angsty<br><br>Genre:<br>boxes that represent a genre, the user can select one of these.<br>Options in set:<br>Grunge<br>Metal<br>Rock<br>Post-Hardcore<br>Punk Rock<br>Pop Punk<br>Goth<br>Shoegaze<br>Indie Rock<br>Emotional Hardcore<br><br>These will be present in the search page and the recommend page.<br>Title: Text Box<br>Artist: Text Box<br><br>These will only be present in the recommend page<br><br>Search: the search button will put whatever was checked and look for songs that match those |

| | |
|---|---|
| | attributes |
| **If Statements** (minimum 3)<br>Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site | **If Statement 1:** Will check to make sure the user does not submit blank song entries into the library.<br>if (name.strip() == '' or artist.strip() == '' or<br>theme == '' or mood == '' or genre == ''):<br>state.submission_message = "Please fill in all fields."<br><br>**If statement 2**: Checks if song.mood == state.search_mood, song.theme == state.search_theme and song.genre == state.search_genre. This will be the if statement that will be inside the loop<br>**If statement 3**: (matching songs is empty list in for loop) Evaluates if not matched_songs to display a custom message instead of an error page.<br>Ex.<br>if len(matching_songs) == 0:<br>state.result_message = "No matching songs found."<br>else:<br>state.result_message = matching_songs[0].name |
| **Loop** (minimum 1)<br>This loop must iterate over the list/dict of dataclass instances in your State attribute.<br>Description of this loop and why it is meaningful to the site | The one loop will Iterate over every Song instance in state.library to check if its .mood .theme and .genre attributes match the active search criteria in State |

# LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.




Drafter Planning Tool (V2.0.0b1) ··· Design Music Website Requirements
Worked for 22s
I think your idea is very workable for an introductory CS project, and it has a lot of potential to satisfy all the requirements without making the application unnecessarily complicated.
I'd recommend developing it as a mood-based music recommendation website. Users select themes and moods, receive song recommendations, browse the song collection, and submit songs of their own.
The key is to make each page and route serve a purpose rather than adding features just to reach the required numbers.
I'll organize the suggestions around your assignment requirements and keep the implementation realistic for a three-week Drafter project.
1. Overall website concept
Possible name: Moodify, Soundtrack, or MoodMatch.
Purpose: Help users discover music that matches their current mood, emotional state, or preferred theme.
For example:
A user selects Happy and Upbeat.
The website searches its song collection.
It returns songs that have matching tags.
The user can save songs to a favorites list or submit a new recommendation.
I would avoid trying to connect to Spotify or Apple Music APIs. You don't need an external music database for this project. Instead, you can create your own small collection of songs using Python dataclasses.
2. Pages and routes
You need at least 7 routes and 5 pages. I recommend having 6 actual pages and 8 routes.
Here is one possible structure:
Page
Route
Purpose
Home
/
Introduces the website and explains how it works
Mood Selection
/mood
Allows users to select moods and themes
Recommendations
/recommendations
Displays songs matching the selected tags
Song Details
/song
Displays information about a selected song
Submit Song
/submit
users to recommend and submit songs

| | | |
|---|---|---|
| Song Library | `/library` | Displays all songs currently stored |
| Favorites | `/favorites` | Displays songs saved by the user |
| About | `/about` | Explains the website and its purpose |

This gives you 8 routes and 8 pages, although some routes could share a page component if necessary.

**One additional idea:** You could have a separate results page for when no songs match the user's selected tags. This would make your application more useful and give you another opportunity to demonstrate conditional statements.

### 3. Dataclasses and State requirements

This is probably the most important part of your design because you can use your existing song idea to satisfy several requirements naturally.

I recommend two dataclasses.

**Dataclass 1: Song**

Each song represents one individual song in your collection.

Python Run

```python
@dataclass
class Song:
    title: str
    artist: str
    genre: str
    tags: list[str]
    description: str
```

For example, a song could look conceptually like this:

| Field | Example |
|---|---|
| title | Golden Hour |
| artist | Example Artist |
| genre | Pop |
| tags | ["Happy", "Romantic", "Relaxing"] |
| description | A relaxing song for a calm afternoon |

Drafter Planning Tool (V2.0.0b1) ··· Design Music Website Requirements

You could use tags such as:

- Happy
- Sad
- Nostalgic
- Romantic
- Energetic
- Relaxing
- Motivational
- Dreamy
- Upbeat
- Melancholic

**Dataclass 2: State**

Your State dataclass could look something like this:

Python Run

```python
@dataclass
class State:
    songs: list[Song]
    selected_tags: list[str]
    favorite_songs: list[Song]
    recommended_songs: list[Song]
    submission_message: str
```

This gives you five fields, and all five have a purpose.

More importantly, you can meaningfully modify at least four of them through different routes.

| State field | Route that modifies it | What happens |
|---|---|---|
| `songs` | Submit Song | Adds a newly submitted Song instance |
| `selected_tags` | Mood Selection | Stores the user's selected moods |
| `favorite_songs` | Song Details | Adds or removes a favorite |
| `recommended_songs` | Recommendations | Stores the matching songs |
| `submission_message` | Submit Song | Updates the confirmation or error message |

Meets the State requirements

You have five fields, a list of class instances, and meaningful modifications to all five.

One important detail: I recommend storing the actual `Song` instances in your favorites list rather than just storing song titles. That makes the relationship between your two dataclasses more apparent.

### 4. Input fields

You need at least three input fields. Your project could naturally include five.

| Input field | Type | Purpose |
|---|---|---|
| Mood selection | CheckBox | Select moods such as Happy or Sad |
| Theme selection | CheckBox | Select themes such as Romantic or Motivational |
| Song title | TextBox | Enter the submitted song's title |
| Artist name | TextBox | Enter the artist |
| Genre | SelectBox | Choose the song's genre |
| Song description | TextArea | Explain why the song fits the selected tags |

You don't necessarily need all six. However, having these inputs makes your application feel more complete.

**My recommendation:** Use checkboxes for moods and themes because users should be able to select multiple tags simultaneously.

For example, someone could select:

- Happy
- Nostalgic
- Relaxing

Your program would then look for songs matching those tags.

### 5. Meaningful if statements

You need at least three meaningful conditional statements. Your application gives you several natural opportunities.

Here are five possibilities.

**If statement 1: Check whether the user selected any tags.**

Python Run

```python
if len(state.selected_tags) == 0:
    # Ask the user to select at least one mood
```

Drafter Planning Tool (V2.0.0b1) ··· Design Music Website Requirements

This prevents the recommendation system from running without input.

**If statement 2: Check whether a song matches.**

Python Run

```python
if match_count > 0:
    # Add the song to the recommendation list
```

This determines whether a song should appear in the recommendations.

**If statement 3: Check whether recommendations exist.**

Python Run

```python
if len(state.recommended_songs) == 0:
    # Display a message saying no songs were found
```

This provides a useful alternative when there are no matches.

**If statement 4: Check whether a song is already a favorite.**

Python Run

```python
if selected_song not in state.favorite_songs:
    # Add the song to favorites
```

This prevents duplicate favorites.

**If statement 5: Validate song submissions.**

```python
if title == "" or artist == "":
    # Display an error message
```

This prevents users from submitting incomplete songs.

You only need three, but these are all useful and directly connected to your application's purpose.

### 6. Meaningful loop

This requirement is especially easy to satisfy with your Song dataclass.

Your loop should iterate through the `songs` list and examine each song's tags.

Conceptually, the recommendation algorithm would work like this:

1. The user selects several tags.
2. Your program loops through every Song instance in `state.songs`.
3. For each song, it loops through the song's tags.
4. It checks whether those tags match the user's selected tags.
5. It counts how many tags match.
6. It adds songs with matching tags to `state.recommended_songs`.

For example:

```python
for song in state.songs:
    match_count = 0

    for tag in song.tags:
        if tag in state.selected_tags:
            match_count += 1

    if match_count > 0:
        state.recommended_songs.append(song)
```

This is a particularly good fit for your assignment because it demonstrates:

- Iterating through a list of class instances.
- Accessing attributes of a dataclass.
- Nested loops.

- Conditional statements.
- Comparing user input against stored data.

You could eventually improve the recommendation system by prioritizing songs with more matching tags, but I would start with this simpler approach.

## 7. Additional features to make your website interesting

Here are some features I think would make your project stand out without making it too complicated.



**1. Song of the Day**

Display one randomly selected song from your collection on the homepage. This gives users a reason to visit even when they don't have a particular mood in mind.



**2. Favorite Songs**

Users can save songs they enjoy and view them on a separate page. This also gives your favorites State field a meaningful purpose.



**3. Why This Song?**

When a recommendation appears, explain why it was selected.

For example: "Recommended because it matches your Happy and Nostalgic tags."



**4. Browse by Genre**

Allow users to browse songs by genre, such as Pop, Rock, Indie, or R&B. This gives your library page additional functionality.

I'd prioritize the favorites feature and the explanation of why a song was recommended.

## 8. How everything fits the assignment

Here is a checklist showing how your proposed application satisfies the requirements.

| Assignment requirement | Your implementation |
| --- | --- |
| At least 7 routes | 8 routes |
| At least 5 pages | 8 pages |
| State dataclass with 4+ fields | 5 fields |

| | |
|---|---|
| List/dictionary of class instances | `songs: list[Song]` |
| Four modified State fields | Songs, selected tags, favorites, recommendations |
| At least 3 input fields | CheckBox, TextBox, SelectBox, TextArea |
| At least 3 if statements | Input validation, matching, favorites, empty results |
| At least 1 meaningful loop | Loop through songs and their tags |
| Legitimate purpose | Mood-based music discovery |
| No global variables | Keep mutable application data in State |
| Drafter library | Build all pages and routes using Drafter |

## 9. Important technical considerations

There are a few things I would keep in mind before starting.

**1. Don't make the music database too large.**

Start with approximately 15–25 songs that you manually enter into your initial State. Give each song 2–4 tags.

You don't need hundreds of songs to demonstrate that your recommendation system works.

**2. Be careful with song submissions.**

Because Drafter runs in the browser using Pyodide and doesn't have a backend or database, submitted songs won't automatically be shared with other users.

You can keep submitted songs in your application's State while it runs. If you want them to persist across browser sessions, you could investigate browser localStorage, but that is an additional technique that you would need to document and understand.

**3. Avoid complicated recommendation algorithms.**

You don't need artificial intelligence or machine learning to recommend songs. Comparing tags and counting matches is enough to create a functional recommendation system.

**4. Keep the interface simple.**

Drafter is a Python frontend library, so focus on making functional pages with clear navigation, buttons, input fields, and song information rather than attempting to build an extremely complicated music-streaming interface.

## 10. My suggested final project scope

**Design Music Website Requirements**

Free offer

If I were planning this as a three-week introductory CS project, I would divide it into three stages.

1. **Week 1: Build the foundation**
   - Create the Song and State dataclasses.
   - Populate the initial song collection.
   - Build the homepage.
   - Create the mood selection page.
   - Implement tag selection.
2. **Week 2: Build the functionality**
   - Implement the recommendation algorithm.
   - Create the recommendation results page.
   - Create the song details page.
   - Implement favorites.
   - Add the song submission form.
3. **Week 3: Finish and test**
   - Build the library and About pages.
   - Complete all eight routes.
   - Test empty inputs and songs without matches.
   - Test adding songs and favorites.
   - Improve the appearance and navigation.

**One final recommendation: Make the project about music discovery, not music streaming.**

This distinction is important. You're building a website that helps people discover songs, not a website that plays or streams music.

That keeps the technical requirements manageable and gives your project a clear, legitimate purpose.

Your uploaded project planning worksheet also asks you to explain the problem, IPO model, data abstraction, challenges, and project scope. Your idea fits those sections quite naturally.

**I think the strongest version of your project is one where users select moods and themes, receive ranked song recommendations, learn why those songs match, and contribute their own songs to the collection.**

That is a cohesive application rather than a collection of unrelated features.

Design Music Website Requirements

Copy of CS1 Final Project 1_ Design(1)....
PDF

The following is a worksheet that is adjacent to the project. I have filled it out to the best of my ability with the ideas that I already had in mind and want to use. I do not want more than the pages listed, unless it is necessary to have more

After reviewing your worksheet, I would actually revise my previous suggestions quite a bit. You already have a clear project concept, and I think it would be better to **build around the five pages you originally planned rather than introduce additional pages like Favorites, Song Library, or Song Details as separate destinations.**

Your original idea is more focused, and it is entirely possible to satisfy the assignment requirements without expanding it significantly.

There are just a few things I'd adjust to make sure your existing design meets the requirements.

## 1. Your original five pages

I would keep exactly these five pages:



**1. Home Page**

Welcome message and three buttons:

- Search
- Recommend
- About



**2. Search Page**

Users select checkboxes for their preferred theme, mood, and genre.



**3. Song Page**

Displays the recommended song, its artist, genre, theme, mood, and album cover.



**4. Recommendation Page**



**5. About Page**

Explains the website, its purpose, and the music it focuses on.

I would not add any more pages.

## 2. How to get at least 7 routes with only 5 pages

This is the one requirement that needs a little creativity.

Remember that routes are not necessarily the same thing as pages. You can have multiple routes that lead to the same page but perform different actions.

Here is what I recommend:

| Route function | Destination | Purpose |
|---|---|---|
| `home` | Home | Displays homepage |
| `search` | Search | Displays search filters |
| `find_song` | Song | Processes selected filters and finds a song |
| `song` | Song | Displays the search result |
| `recommend` | Recommendation | Displays submission form |
| `submit_song` | Recommendation | Processes the submitted song |
| `about` | About | Displays information about the website |

7 routes, 5 pages

The distinction is that `find_song` and `submit_song` are processing routes. They don't need to introduce new pages.

For example, `find_song` can process the user's selected checkboxes and then navigate to the existing Song page.

Similarly, `submit_song` can process the submitted information and return the user to the existing Recommendation page with a confirmation message.

This gives you seven meaningful routes without introducing unnecessary destinations.

## 3. Your dataclasses

I would keep your existing Song dataclass almost exactly as you designed it.

Drafter Planning Tool (V2.0.0b1) ··· Design Music Website Requirements

**Song dataclass**

Python Run

```python
@dataclass
class Song:
    name: str
    artist: str
    theme: str
    mood: str
    genre: str
```

This is appropriate because your project revolves around matching songs based on these five characteristics.

For your State dataclass, I suggest:

**State dataclass**

Python Run

```python
@dataclass
class State:
    songs: list[Song]
    selected_theme: str
    selected_mood: str
    selected_genre: str
    current_song: Song
    submission_message: str
```

Here is how these attributes would function:

| Attribute | Purpose | How it changes |
|---|---|---|
| `songs` | Stores all available songs | New songs are added through submissions |
| `selected_theme` | Stores selected theme | Updated on the Search page |
| `selected_mood` | Stores selected mood | Updated on the Search page |
| `selected_genre` | Stores selected genre | Updated on the Search page |
| `current_song` | Stores the matched song | Updated when a search is processed |
| `submission_message` | Stores submission feedback | Updated when a song is submitted |

This gives you six State attributes, including the required list of Song instances.

It also gives you more than four attributes that are meaningfully modified.

**One consideration:** Your assignment says the theme, mood, and genre checkboxes can be left unchecked. You should account for that in your State design. Empty strings (`""`) could represent no selection.

### 4. Input fields: Keep your existing design

Your worksheet already identifies three categories of checkboxes:

- Theme
- Mood
- Genre

I would keep these exactly as they are.

There is one important detail, though: you should make sure that your checkbox implementation actually allows the user to select one option from each category, or leave the category unanswered.

For your Recommendation page, you could also use:

- TextBox for song name.
- TextBox for artist.
- Checkboxes for theme, mood, and genre.

This gives you more than the required three input fields without changing your original concept.

### 5. If statements

I would use these four conditional statements:

| If statement | What it evaluates | What happens |
|---|---|---|
| 1 | Whether the user selected a theme, mood, or genre | Determines which filters should be applied |
| 2 | Whether a song matches the selected attributes | Determines whether the song is eligible |
| 3 | Whether a matching song was found | Displays the song or a no-results message |
| 4 | Whether the submitted song has a name and artist | Accepts or rejects the submission |

These are all directly related to your existing pages.

### 6. Loop

Your loop can be very straightforward.

When someone searches for a song, your program will iterate through `state.songs`.

Design Music Website Requirements

For each Song instance, it will compare the song's attributes against the user's selected attributes.

For example, conceptually:

Python Run

```
for song in state.songs:
    if song.theme == state.selected_theme:
        # Check the remaining selected attributes
```

You will need to account for unchecked categories, so the comparison should only apply filters that the user actually selected.

One important design decision: Since you have three independent filters, you should decide whether a song needs to match **all selected attributes** or just one of them.

I recommend requiring all selected attributes to match, while ignoring any categories the user left blank. This makes the recommendations more precise.

## 7. One change I would make to your original design

Your worksheet currently says:

> The theme will not play a role in what song is given to them.

The same wording appears under Mood and Genre.

I suspect this is leftover wording rather than your intended functionality, because your website is specifically designed to recommend songs based on these attributes.

I would change this to:

- Theme: Helps determine which songs are recommended.
- Mood: Helps determine which songs are recommended.
- Genre: Helps determine which songs are recommended.

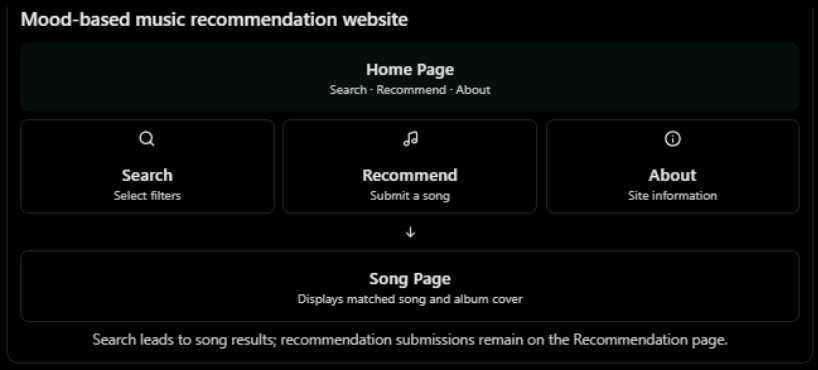
This makes the purpose of your application much clearer.

---

## 8. Final recommended structure

Here is how I would organize the entire application.

Music Website Requirements




### My final thoughts

I actually prefer your original design over the expanded version I suggested earlier. It has a more coherent purpose and is more manageable for an introductory programming project.

I would make only three major adjustments:

1. Add two processing routes ( `find_song` and `submit_song` ) to meet the route requirement.
2. Expand your State dataclass to include the song collection, selected filters, current song, and submission feedback.
3. Make sure all three filters actually influence song recommendations.

**One question worth deciding before you start coding:** Do you want the search to return one randomly selected matching song, or display every song that matches the user's selected theme, mood, and genre?

Both are achievable with your current design, but they lead to slightly different implementations.

Drafter Planning Tool (V2.0.0b1) ··· Design Music Website Requirements

this is the finished table, i chnaged some things. what do you think? requirements.

Requirement
Implementation
Purpose/Functionality

A 1-2 sentence description of the purpose or functionality of your site
The purpose of this website is to provide users a tool to get a song recommendation based on a theme or mood they have in mind. This website will focus on alternative songs, but different genres in that umbrella will be covered.
State Dataclass Attributes

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.
@dataclass
class State:
library:list[Song]

- The library attribute is a list of songs (Datclass Song) that the program has stored which will be used for matching inputs with outputs. This will be modified through the add_song route

search_theme:str

- The search_theme attribute is the current theme selected by the user. It is modified through the perform_search route

search_mood:str

- The search_mood attribute is the current moods selected by the user. It is modified through perform_search route

search_genre:str

- The search_genre attribute is the current genre selected by the user. It is modified through the perform_search route.

1) ··· Design Music Website Requirements

recommendations: int

- Recommendations is an attribute pertaining to the total number of songs recommended by the user. It is modified through the add_song route.

Second Dataclass

Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)
@dataclass
class Song:
name:str
artist: str
theme: str
mood:str
genre: str
Some of these will be already defined in the code but new ones may be made through the add_song route.
Routes (minimum 7)

The names of your route functions
/index - Displays the home page that will have navigation buttons.
/search- Displays the search page which allows the users to choose their inputs.
/result- Displays the result page
/recommend- Displays the recommend page
/perform_search - Updates the values for state.search_theme, state.search_mood, and state.search_genre based on the boxes that the user checked
/add_song - this adds (appends) a new Song instance to state.library
/about - displays the about page
Pages (minimum 5)

A brief description of the pages in your site
Home: The home page will be the very first page the user will see. It will have some text welcoming the user to the site and three buttons. The buttons will be "Search", "Recommend", and "About"

Search Page: The search page is the page in which the user will be

given different check boxes they can mark depending on what song they are looking for. After checking specific attributes, the user will then click

Song Page: The song page is the page that will be given to the user as an output in response to their input, it will contain a song recommended based on their matched input and the image of the album cover the song belongs to.

Recommendation Page:  The recommendation page is the page that will result if the user presses the recommend button on the home page. In this page a user is able to input a song. There will be a title box in which the title of the song will be in and then checkboxes that the user will have to click to indicate the song's genre, mood, and theme.

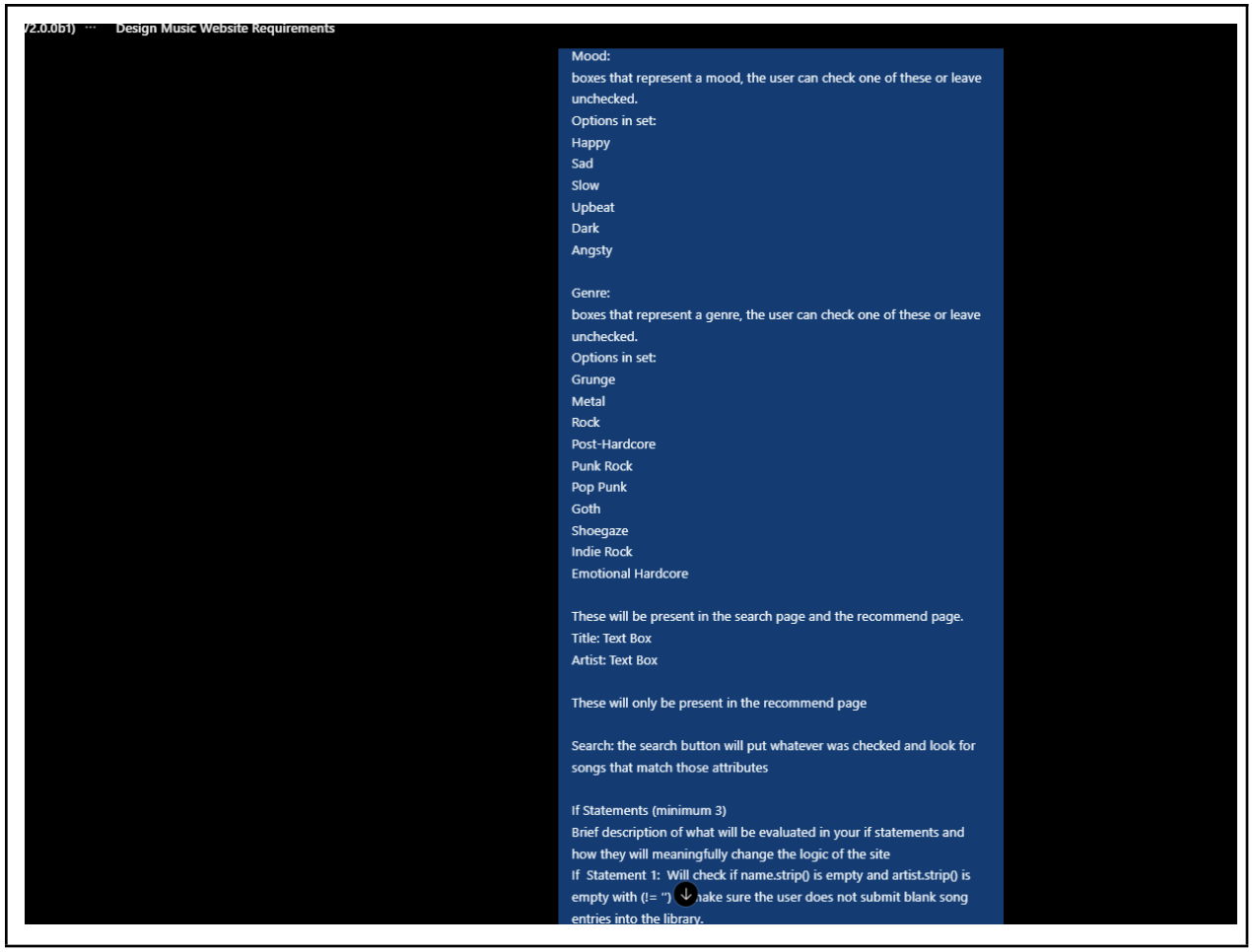
About Page:The about page will be the page that will pop up when the user clicks the about button, it will simply contain text explaining how the website came to be and what it is about

*all these pages will have a home button to return to the home page

User Input (minimum 3)

Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes

For more information on these input types, see Drafter Documentation.
The way the user will put input is through a series of checkboxes.
*One of the following must be checked
Theme:
boxes that represent a theme, the user can check one of these or leave unchecked.
Options in set:
Romantic
Heartbreak
Rebellion
Isolation
Self-Discovery
Death


/2.0.0b1) ··· Design Music Website Requirements
Mood:
boxes that represent a mood, the user can check one of these or leave unchecked.
Options in set:
Happy
Sad
Slow
Upbeat
Dark
Angsty
Genre:
boxes that represent a genre, the user can check one of these or leave unchecked.
Options in set:
Grunge
Metal
Rock
Post-Hardcore
Punk Rock
Pop Punk
Goth
Shoegaze
Indie Rock
Emotional Hardcore
These will be present in the search page and the recommend page.
Title: Text Box
Artist: Text Box
These will only be present in the recommend page
Search: the search button will put whatever was checked and look for songs that match those attributes
If Statements (minimum 3)
Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site
If Statement 1: Will check if name.strip() is empty and artist.strip() is empty with (!= '') make sure the user does not submit blank song entries into the library.

Tool (V2.0.0b1) ··· Design Music Website Requirements

If statement 2: Checks if song.mood == state.search_mood, song.theme == state.search_theme and song.genre == state.search_genre. This will be the if statement that will be inside the loop
If statement 3: Evaluates if len(matching_songs) == 0: to display a custom message instead of an error page.

Loop (minimum 1)
This loop must iterate over the list/dict of dataclass instances in your State attribute.
Description of this loop and why it is meaningful to the site
The one loop will Iterate over every Song instance in state.song_library to check if its .mood .theme and/or .genre attributes match the active search criteria in State

Show less

I think this is a significant improvement. Your design is now much more cohesive, and you've managed to keep the original five pages while making the requirements fit naturally into the functionality.

I would say you're very close to having a solid project plan. However, there are **four things I would fix or clarify before submitting this table**, particularly regarding your State dataclass and recommendation logic.

### 1. State dataclass: One important change

Your current State dataclass:

Python Run

```python
@dataclass
class State:
    library: list[Song]
    search_theme: str
    search_mood: str
    search_genre: str
    recommendations: int
```

This is a good structure, but I have one concern about `recommendations`.

You describe it as:

> An attribute pertaining to the total number of songs recommended by the user.

This sounds like a counter that increases whenever someone submits a song. However, it isn't entirely clear

Planning Tool (V2.0.0b1) ··· Design Music Website Requirements

Free offer

how counting submitted songs contributes to your website's functionality.

I'd suggest keeping the counter but making its purpose more specific.

For example:

- `library`: Stores all available songs. Modified through `add_song`.
- `search_theme`: Stores the selected theme. Modified through `perform_search`.
- `search_mood`: Stores the selected mood. Modified through `perform_search`.
- `search_genre`: Stores the selected genre. Modified through `perform_search`.
- `recommendations`: Tracks how many songs users have contributed to the library. Modified through `add_song`.

This makes the fifth attribute meaningful.

State requirement satisfied

You have five attributes, including a list of Song instances, and all five can be meaningfully modified.

One thing to remember: your `perform_search` route should actually assign the selected values to State, not just use them temporarily.

### 2. Routes: Your seven routes are good

I like your current route structure. You have exactly seven routes, and you don't need additional pages.

| Route | Functionality |
|---|---|
| `/index` | Displays homepage |
| `/search` | Displays search form |
| `/result` | Displays matching song |
| `/recommend` | Displays song submission form |
| `/perform_search` | Processes search inputs |
| `/add_song` | Adds a new Song to the library |
| `/about` | Displays information about the website |

There is one small distinction I'd make:

- `/perform_search` should process the input and determine which songs match.
- `/result` should display the result of that processing.

This is a sensible separation of responsibilities.

Also, make sure your processing routes actually return Drafter pages or navigate to pages appropriately. They shouldn't simply perform an action and leave the user without a response.

### 3. User input: I like your checkbox system

Your input design makes sense, especially because you're focusing on alternative music.

You currently have:

- Theme checkboxes
- Mood checkboxes
- Genre checkboxes
- Song title TextBox
- Artist TextBox

That's five categories of input, which comfortably exceeds the minimum.

However, I noticed something important.

You wrote:

> One of the following must be checked.

I would clarify whether this applies to each category or to the entire search.

My recommendation:

**Search page:** The user must select at least one option across all three categories.

For example:

- Theme: Heartbreak
- Mood: Sad
- Genre: Rock

Or they could select just:

- Mood: Sad

Both should be valid searches.

**Recommendation page:** Require a title, artist, and one selection from each of the three categories.

This will help ensure that the songs submitted to your library have enough information to be useful.

### 4. If statements: Three good choices, but adjust the first

Your three if statements are appropriate. I would just ke a small correction to the first one.

**Design Music Website Requirements**

You wrote:

```python
if name.strip() != '' and artist.strip() != '':
```

This checks that both fields contain something.

However, you'll want to consider what happens when either field is empty.

For example:

```python
if name.strip() == '' or artist.strip() == '':
    # Display an error message
```

Otherwise:

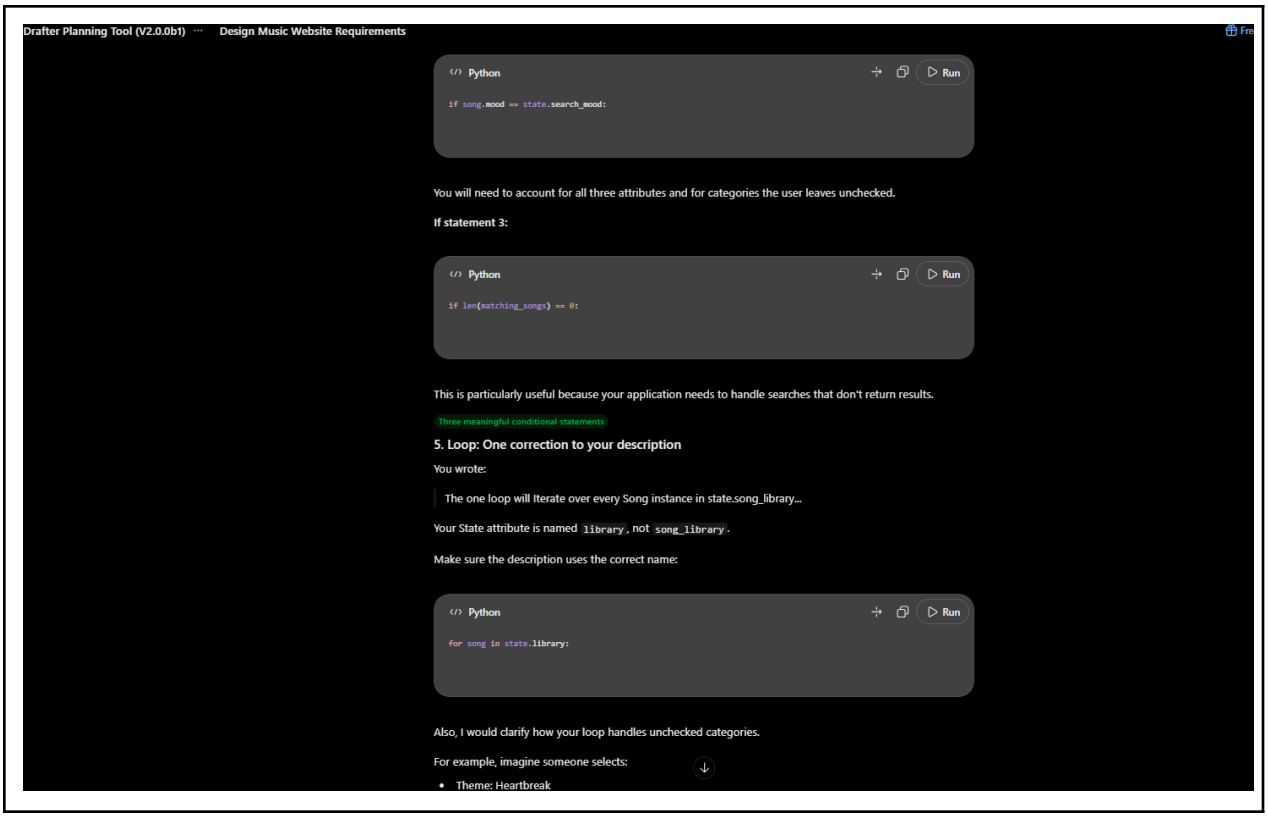
```python
if name.strip() != '' and artist.strip() != '':
    # Add the song
```

Both approaches work.

Your second and third if statements are also good:

**If statement 2:**

```python
if song.mood == state.search_mood:
```


Drafter Planning Tool (V2.0.0b1) ··· Design Music Website Requirements
Python
Run
if song.mood == state.search_mood:
You will need to account for all three attributes and for categories the user leaves unchecked.
If statement 3:
Python
Run
if len(matching_songs) == 0:
This is particularly useful because your application needs to handle searches that don't return results.
Three meaningful conditional statements
5. Loop: One correction to your description
You wrote:
The one loop will Iterate over every Song instance in state.song_library...
Your State attribute is named library, not song_library.
Make sure the description uses the correct name:
Python
Run
for song in state.library:
Also, I would clarify how your loop handles unchecked categories.
For example, imagine someone selects:
Theme: Heartbreak

- Mood: Sad
- Genre: (nothing selected)

Your program should still return songs that match Heartbreak and Sad, regardless of genre.

Conceptually, the matching logic should follow this principle:

- If a theme was selected, compare the song's theme.
- If a mood was selected, compare the song's mood.
- If a genre was selected, compare the song's genre.
- Ignore categories that weren't selected.

This makes the recommendation system considerably more flexible.

---

## 6. My biggest recommendation: Decide how song matching works

There is one design decision that I think you should settle before coding.

Suppose a user searches for:

| Category | Selection |
|---|---|
| Theme | Heartbreak |
| Mood | Sad |
| Genre | Rock |

Your program finds three songs:

| Song | Theme | Mood | Genre |
|---|---|---|---|
| Song A | Heartbreak | Sad | Rock |
| Song B | Heartbreak | Sad | Metal |
| Song C | Heartbreak | Happy | Rock |

Which songs should be returned?

I recommend **Song A only**, because it matches all three selected attributes.

If the user leaves Genre blank, however, both Song A and Song B should qualify.

This is a straightforward matching system and doesn't require any advanced algorithms.

## 7. Final assessment

Here's where I think your project currently stands:

| Requirement | Assessment |
| --- | --- |
| Legitimate purpose | Satisfied |
| State with 4+ fields | Satisfied |
| List of dataclass instances | Satisfied |
| Four meaningfully modified fields | Satisfied |
| Second dataclass | Satisfied |
| Seven routes | Satisfied |
| Five pages | Satisfied |
| Three input fields | Satisfied |
| Three if statements | Satisfied |
| Meaningful loop | Satisfied, with clarification needed |

**Overall, I think you have a very solid project design.**

I would not add any more pages or major features. Your existing five pages are enough, and your seven routes have distinct responsibilities.

The only things I'd finalize before moving on are the behavior of your `recommendations` counter, the handling of unchecked categories, and the exact matching rules.

Those are relatively small decisions, and once they're settled, I think you're ready to start implementing the Drafter application.

Design Music Website Requirements

here is updated :

| | |
|---|---|
| **Purpose/Functionality**<br>**A 1-2 sentence description of the purpose or functionality of your site** | **The purpose of this website is to provide users a tool to get a song recommendation based on a theme or mood they have in mind. This website will focus on alternative songs, but different genres in that umbrella will be covered.** |
| **State Dataclass Attributes**<br><br>Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.<br><br>Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site. | @dataclass<br>class State:<br>library:list[Song] |

- The library attribute is a list of songs (Datclass Song) that the program has stored which will be used for matching inputs with outputs. This will be modified through the add_song route

search_theme:str

- The search_theme attribute is the current theme selected by the user. It is modified through the perform_search route

search_mood:str

- The search_mood attribute is the current moods selected by the user. It is modified through perform_search route

search_genre:str

- The search_genre attribute is the current genre selected by the user. It is modified through the perform_search route.

result_message:

- attribute pertaining to the message given to the user after a search has been made, will appear in result page

submission_message: str

- attribute pertaining to the message given to the user after the submission in recommendation has been made

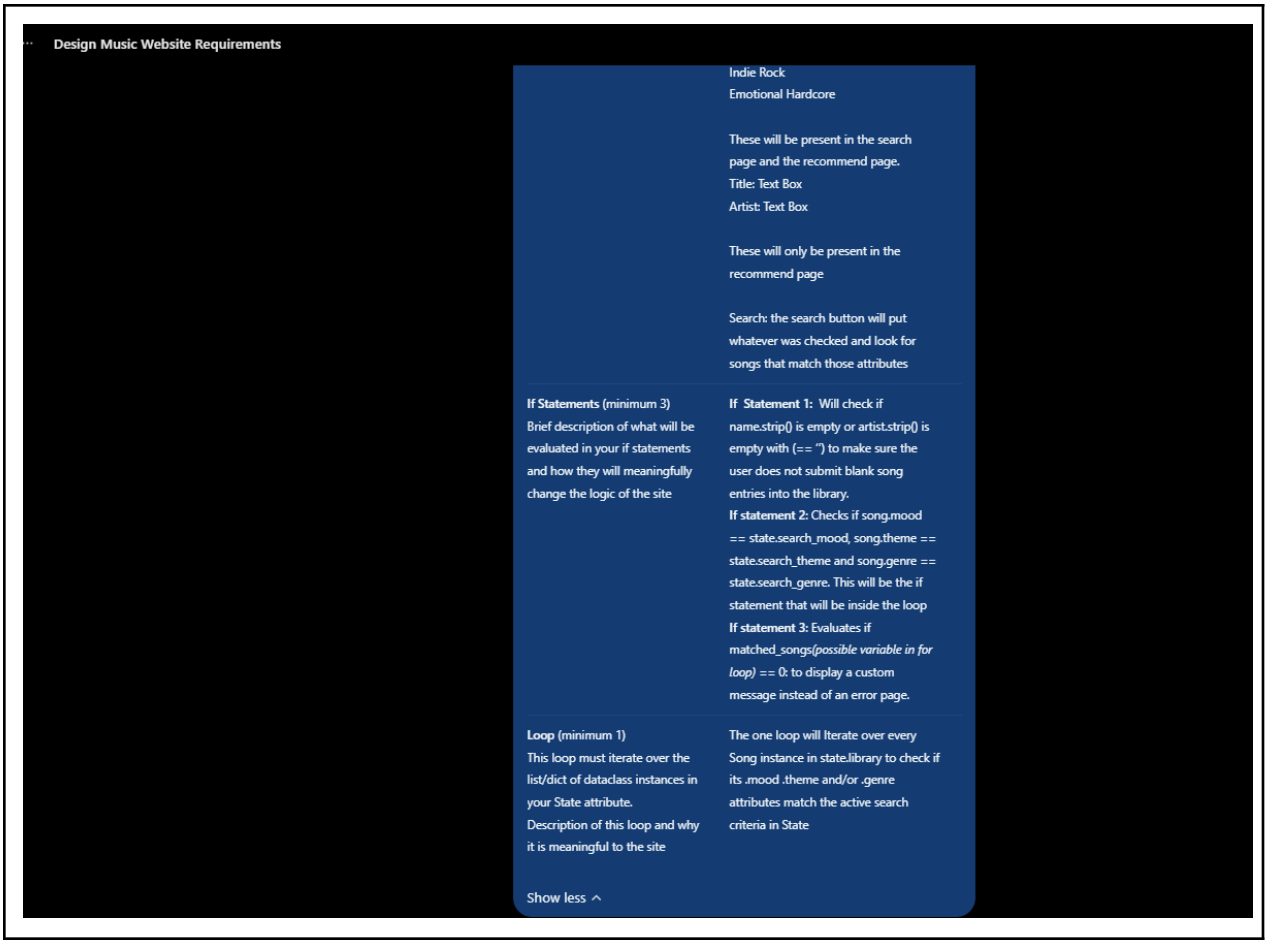
| | |
|---|---|
| **Second Dataclass**<br><br>**Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)** | **@dataclass**<br>**class Song:**<br>**name:str**<br>**artist: str**<br>**theme: str**<br>**mood:str**<br>**genre: str**<br>**Some of these will be already defined in the code but new ones may be made through the add_song route.** |
| **Routes** (minimum 7)<br><br>The names of your route functions | index(state: State) - Displays the home page that will have navigation buttons.<br>search(state: State) - Displays the search page which allows the users to choose their inputs.<br>result(state: State) - Displays the result page<br>recommend(state: State) - Displays the recommend page<br>perform_search(state: State, theme: str, mood: str, genre: str) - Updates the values for state.search_theme, state.search_mood, and state.search_genre based on the boxes that the user checked<br>add_song(state: State, name: str, artist: str, theme: str, mood: str, genre: str) - this adds (appends) a new Song instance to state.library<br>about_page(state: State) - displays the about page |

**Design Music Website Requirements**

| | |
|---|---|
| Pages (minimum 5)<br><br>A brief description of the pages in your site | Home: The home page will be the very first page the user will see. It will have some text welcoming the user to the site and three buttons. The buttons will be "Search", "Recommend", and "About"<br><br>Search Page: The search page is the page in which the user will be given different select boxes they can mark depending on what song they are looking for. After selecting specific attributes, the user will then click search song<br><br>Song Page: The song page is the page that will be given to the user as an output in response to their input, it will contain a song recommended based on their matched input and the image of the album cover the song belongs to.<br><br>Recommendation Page: The recommendation page is the page that will result if the user presses the recommend button on the home page. In this page a user is able to input a song. There will be a title box in which the title of the song will be in and then checkboxes that the user will have to click to indicate the song's genre, mood, and theme.<br><br>About Page:The about page will be the page that will pop up when the user clicks the about button, it will simply contain text explaining how the website came to be and what it is about |

Design Music Website Requirements

| | |
|---|---|
| | *all these pages will have a home button to return to the home page |
| **User Input** (minimum 3)<br><br>Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes<br><br>For more information on these input types, see **Drafter Documentation.** | The way the user will put input is through a series of selection boxes.<br>The following must be selected from a dropdown menu<br>Theme:<br>boxes that represent a theme, the user can check one of these.<br>Options in set:<br>Romantic<br>Heartbreak<br>Rebellion<br>Isolation<br>Self-Discovery<br>Death<br><br>Mood:<br>boxes that represent a mood, the user can select one of these.<br>Options in set:<br>Happy<br>Sad<br>Slow<br>Upbeat<br>Dark<br>Angsty<br><br>Genre:<br>boxes that represent a genre, the user can select one of these.<br>Options in set:<br>Grunge<br>Metal<br>Rock<br>Post-Hardcore<br>Punk Rock<br>Pop Punk<br>Goth<br>Shoegaze |


Design Music Website Requirements
Indie Rock
Emotional Hardcore
These will be present in the search page and the recommend page.
Title: Text Box
Artist: Text Box
These will only be present in the recommend page
Search: the search button will put whatever was checked and look for songs that match those attributes
If Statements (minimum 3)
Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site
If Statement 1: Will check if name.strip() is empty or artist.strip() is empty with (== '') to make sure the user does not submit blank song entries into the library.
If statement 2: Checks if song.mood == state.search_mood, song.theme == state.search_theme and song.genre == state.search_genre. This will be the if statement that will be inside the loop
If statement 3: Evaluates if matched_songs(possible variable in for loop) == 0: to display a custom message instead of an error page.
Loop (minimum 1)
This loop must iterate over the list/dict of dataclass instances in your State attribute.
Description of this loop and why it is meaningful to the site
The one loop will Iterate over every Song instance in state.library to check if its .mood .theme and/or .genre attributes match the active search criteria in State
Show less

Design Music Website Requirements

I think this version is considerably stronger. You've addressed most of the issues from the previous draft, particularly by replacing the recommendations counter with `result_message` and `submission_message`.

I also like that you've switched to dropdown menus (SelectBoxes). They make sense for your design because users are selecting one option per category rather than multiple options.

**I would keep your five pages and seven routes exactly as they are.** You don't need to add anything else.

There are, however, a few things I'd fix before considering the table finished.

### 1. State dataclass — Almost there

Your current State has six attributes:

| Attribute | Type | Purpose |
|---|---|---|
| `library` | `list[Song]` | Stores all available songs |
| `search_theme` | `str` | Stores the selected theme |
| `search_mood` | `str` | Stores the selected mood |
| `search_genre` | `str` | Stores the selected genre |
| `result_message` | `str` | Stores the search result message |
| `submission_message` | `str` | Stores the submission confirmation or error |

I recommend explicitly adding `: str` to `result_message`.

Your State definition would therefore look like:

Python

Run

```python
@dataclass
class State:
    library: list[Song]
    search_theme: str
    search_mood: str
    search_genre: str
    result_message: str
    submission_message: str
```

Six State attributes

One thing to be careful about: your assignment requires at least four State attributes to be meaningfully modified in routes.

You have a natural way to accomplish this:

- `library` → modified by `add_song`.
- `search_theme` → modified by `perform_search`.
- `search_mood` → modified by `perform_search`.
- `search_genre` → modified by `perform_search`.
- `result_message` → modified by `perform_search`.
- `submission_message` → modified by `add_song`.

This satisfies that requirement comfortably.

### 2. Second dataclass — Good as written

I wouldn't change your Song dataclass.

Python Run

```python
@dataclass
class Song:
    name: str
    artist: str
    theme: str
    mood: str
    genre: str
```

It contains exactly the information your recommendation system needs.

One small suggestion: When adding songs, make sure you create a new Song instance and append that instance to `state.library`, rather than appending individual strings.

### 3. Routes — Keep all seven

Your routes are well organized.

| Route | Responsibility |
| --- | --- |
| `index(state)` | Displays homepage |
| `search(state)` | Displays search form |
| `result(state)` | Displays search results |

Design Music Website Requirements

| | |
|---|---|
| `recommend(state)` | Displays submission form |
| `perform_search(state, theme, mood, genre)` | Processes search |
| `add_song(state, name, artist, theme, mood, genre)` | Adds submitted song |
| `about_page(state)` | Displays About page |

One detail worth remembering: `perform_search` and `add_song` should return a page or otherwise navigate to an appropriate existing page.

For example:

- `perform_search` processes the search and leads to `result`.
- `add_song` processes the submission and returns to `recommend`, where `submission_message` is displayed.

This lets you keep your seven routes without creating additional pages.

### 4. User input — One thing to clarify

Your dropdown menu design is a good choice.

You have:

- Theme: SelectBox
- Mood: SelectBox
- Genre: SelectBox
- Song title: TextBox
- Artist: TextBox

Five input fields

However, I noticed a slight inconsistency.

You wrote:

> The following must be selected from a dropdown menu.

And later:

> The user can select one of these.

I would clarify whether all three categories are mandatory.

My recommendation is:

- Search page: All three categories can be left blank, but at least one must be selected.
- Recommendation page: All three categories must be selected.

This gives users more flexibility when searching while ensuring submitted songs have complete information.

Also, since you're using SelectBoxes, change references to "checkboxes" elsewhere in the worksheet to "dropdown menus" or "SelectBoxes."

### 5. If statements — Two small corrections

Your first conditional is good:

```python
if name.strip() == '' or artist.strip() == '':
```

This checks whether either required text field is empty.

Your second conditional needs a little more thought.

Currently, you have:

```python
if song.mood == state.search_mood and song.theme == state.search_theme and song.genre == state.search_genre:
```

This works if all three categories are mandatory.

However, if you allow users to leave categories blank, you'll need to account for that.

For example, if someone selects only `Sad` as their mood, your program shouldn't reject songs simply because their theme and genre aren't selected.

You can describe this in your worksheet as:

**If Statement 2:** Checks whether a song matches the user's selected mood, theme, and genre. Only selected categories will be used when determining whether a song matches.

Design Music Website Requirements

Your third conditional also needs a small correction.

You wrote:

```python
if matched_songs == 0:
```

If `matched_songs` is a list, this should be:

```python
if len(matched_songs) == 0:
```

This checks whether the list contains any matching songs.

You could also write:

```python
if not matched_songs:
```

But the first version is probably easier to understand for your class.

### 6. Loop — Correct

Your loop description is good:

> The one loop will iterate over every Song instance in state.library to check if its .mood, .theme and/or .genre attributes match the active search criteria in State.

I would only make one small wording adjustment:

**Design Music Website Requirements**

"The loop iterates over every Song instance in `state.library`, comparing its mood, theme, and genre attributes against the user's selected search criteria. Matching songs are added to a separate list, which is then used to determine the search results."

This is a little more specific about what the loop actually accomplishes.

---

### 7. Final requirement checklist

Here's how I would evaluate your current design.

| Requirement | Status |
|---|---|
| Legitimate purpose | Satisfied |
| State dataclass with 4+ fields | Satisfied |
| List of second dataclass instances | Satisfied |
| Four meaningfully modified State fields | Satisfied |
| Second dataclass | Satisfied |
| At least 7 routes | Satisfied |
| At least 5 pages | Satisfied |
| At least 3 input fields | Satisfied |
| At least 3 meaningful if statements | Satisfied |
| Meaningful loop | Satisfied |

### My overall opinion

Strong project design

I think you're at the point where you should stop adding features and start thinking about implementation.

Your project has a clear purpose, your dataclasses are appropriate, and your routes have distinct responsibilities.

The only decisions I'd finalize before coding are:

1. Whether search categories can be left blank.
2. Whether submitted songs require all three categories.
3. Whether your results page displays one matching song or multiple matching songs.

My recommendation for the third decision is to display **one randomly selected matching song**. That makes

Design Music Website Requirements

the website feel more like a recommendation tool, and it keeps the result page simple.

You have a good foundation here. I wouldn't make any major structural changes.

will my writing work if no attributes can be left blank and all three are needed for submission? Also, what if I just want one song to be returned?

Yes! Your writing will still work, but I would make one small clarification: **all three categories (theme, mood, and genre) are required when someone submits a new song on the Recommend page.** That means your `add_song` route can assume those fields have values, as long as you validate the form before adding the song.

For example, your first `if` statement could check that none of the required fields are blank:

Python Run

```
if (name.strip() == '' or artist.strip() == '' or
    theme == '' or mood == '' or genre == ''):
    state.submission_message = "Please fill in all fields."
```

**What if you only want one song returned?**

That's completely fine! Your loop can still check every song in `state.library`, but instead of displaying every match, you can return just one.

A simple approach for a beginner project is:

1. Create an empty list called `matching_songs`.
2. Loop through `state.library`.
3. Add each song that matches all three selected categories to `matching_songs`.
4. Check whether the list is empty.
5. If it isn't empty, display just the first matching song: `matching_songs[0]`.

For example:

Python Run

```
if len(matching_songs) == 0:
    state.result_message = "No matching songs found."
else:
    state.result_message = matching_songs[0].name
```

You could also display the artist and other song details alongside the title.

**One important distinction:** Requiring all three categories for song submissions doesn't mean users must select all three categories when searching. You can choose to require all three for searches too, which would make your matching logic simpler. If you do that, your project description should say that users select one theme, one mood, and one genre to receive a recommendation.

# Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.